Final Project Report

Farm2Market

An online Farmers Marketplace

Prepared By: Team Forerunners

Group Details:

Name	Student ID
Chowdhury Nabil Ahmed	0242310005341011
MD. Shardul Rahman Turjo	0242310005341263
MD. Sajib Ahammad	0242310005341473

Semester: Spring 2025

Batch: 40 **Section: E-2**

Course Code: 331 **Course Name: Software Engineering Design Capstone Project**

Course Teacher Name: Rahat Uddin Azad

Designation: Lecturer

Submission Date: 16/04/2026

1. Executive Summary

1.1. Project Overview:

Farm2Market is a comprehensive Django-based web application that directly connects farmers with buyers, eliminating middlemen and enabling streamlined agricultural commerce. The platform addresses the critical pain point in traditional agricultural supply chains where farmers lose significant revenue due to intermediaries, and buyers face inflated prices and inconsistent product quality.

The proposed solution is a full-stack e-commerce system with role-based dashboards for both farmers and buyers, featuring real-time inventory management, order tracking, logistics coordination, and multi-user notifications. The platform enables farmers to list products directly and manage orders end-to-end, while buyers can browse, purchase, and track deliveries seamlessly.

Value Proposition:

- **For Farmers**: Direct market access, better pricing control, real-time order management, and reduced distribution costs
- **For Buyers:** Competitive pricing, product freshness, transparent supply chain, and reliable delivery tracking
- **System Impact:** Reduces supply chain costs by 15-25%, increases farmer profitability, and ensures buyer satisfaction through efficiency

1.2. Technology Stack

Layer	Technology
Backend Framework	Django 6.0.3
Database	PostgreSQL (Supabase) / SQLite (Development)
Authentication	Django Built-in Authentication System
Frontend	HTML5, CSS3, Vanilla JavaScript
Image Processing	Pillow 12.2
Environment Management	Environment Management
Python-dotenv	Python-dotenv

Team information:

	Name: **Chowdhury Nabil Ahmed** Student ID: **0242310005341011** Role: **Project Lead, Frontend Developer** Responsibilities: ➢ Project setup, GitHub setup and initial configuration ➢ Base template design and frontend architecture ➢ Assisting backend integration with database and frontend
	Name: **MD Shardul Rahman Turjo** Student ID: **0242310005341263** Role: **Backend Developer** Responsibilities: ➢ Implementation of core backend logic ➢ Development of Dashboard and Profile functionalities ➢ Order lifecycle management (placement to delivery)
	Name: **MD SAJIB AHAMMAD** Student ID: **0242310005341473** Role: **Database & Profile Management** Responsibilities: ➢ Database structure support and maintenance ➢ Development of Buyer and Farmer Profile pages

2. Introduction

Farm2Market is a web-based Farmers Marketplace designed to connect farmers directly with buyers through a simple, secure, and user-friendly online platform. The system aims to reduce dependency on middlemen, ensure fair pricing, and improve market accessibility for farmers in the digital age.

2.1 Problem Statement

The traditional agricultural supply chain involves multiple intermediaries—wholesalers, retailers, and distributors—who take significant margins at each transaction level.

This results in:

- Farmer Revenue Loss: Farmers receive only 30-40% of the final retail price
- Buyer Cost Inflation: Buyers pay 2-3x the farm-gate price
- Quality Degradation: Products deteriorate through multiple transshipments
- Information Asymmetry: Farmers lack market knowledge; buyers lack supply transparency
- Inefficient Logistics: Uncoordinated delivery creates delays and wastage

Farm2Market eliminates these inefficiencies by:

- Creating direct farmer-buyer connections
- Reducing transaction chain length
- Digitizing order and logistics management
- Enabling real-time inventory and delivery tracking

2.2 Project Objectives

The project aims to achieve the following strategic goals:

- **Enable Direct Market Access:** Provide farmers a digital platform to list, manage, and sell products directly to buyers without intermediaries

- **Streamline Order-to-Delivery Workflow**: Implement a complete order lifecycle with status tracking, farmer confirmation, logistics assignment, and delivery verification
- **Implement Role-Based Multi-User System**: Create isolated, secure dashboards for farmers and buyers with role-specific functionalities and access controls
- **Provide Real-Time Inventory & Notification System**: Maintain live product stock, notify users of order status changes, and alert farmers to new orders
- **Establish Scalable Architecture**: Build a modular, maintainable backend with normalized database design supporting future feature expansion

2.3 System Boundaries (Scope)

<u>What the System DOES:</u>

- Allow farmers to register, list products with images, prices, and quantities
- Enable buyers to browse products, add to cart, and checkout by farmer
- Support session-based shopping for unauthenticated users with cart merge on login
- Manage complete order workflow (PENDING → CONFIRMED → DELIVERED → COMPLETED)
- Assign logistics providers to orders and track delivery status
- Generate notifications for order events
- Provide admin panel for category and logistics management
- Support user profile management (farm details for farmers, delivery address for buyers)

<u>What the System DOES NOT:</u>

- Process actual payments
- Handle real-time GPS tracking of deliveries
- Implement bidding/auction features
- Manage recurring/subscription orders
- Provide advanced analytics or reporting dashboards
- Support multiple languages or localization
- Include mobile app (web-only in current version)

2.4 Stakeholder Analysis

Stakeholder	Role	Needs
Farmer	Producer & Seller	List products, manage inventory, confirm orders, assign logistics, track deliveries, receive payments
Buyer	Consumer & Shopper	Browse products, add to cart, checkout, track orders, confirm receipt, manage delivery address
Admin	System Manager	Manage categories, add logistics providers, monitor system health, handle disputes
Project Owner	Stakeholder	Ensure project completion, quality assurance, deployment readiness

3. Requirement Specification

3.1 Functional Requirements (FR)

ID	Feature Name	Description	Priority	Status
FR-1	User Registration	Allow users to register as either Farmer or Buyer with role-specific fields (farm name/location for farmers, delivery address for buyers)	**High**	Implemented
FR-2	User Authentication	Provide secure login/logout functionality using Django's built-in authentication	**High**	Implemented
FR-3	Product Listing (Farmer)	Enable farmers to add new products with image, price, quantity, category, description, and unit	**High**	Implemented
FR-4	Product Management (Farmer)	Allow farmers to edit, view, and delete their products; display stock status dynamically	**High**	Implemented
FR-5	Product Browsing (Buyer)	Allow buyers to view all products with filters by category, search by name, and view product details	**High**	Implemented
FR-6	Shopping Cart	Implement session-based cart for unauthenticated users; database cart for logged-in buyers	**High**	Implemented
FR-7	Cart Management	Enable buyers to add items, modify quantities, remove items, and view cart summary	**High**	Implemented
FR-8	Cart Merge on Login	Automatically merge session cart into database cart upon user login	**High**	Implemented
FR-9	Checkout Process	Group ordered items by farmer; create separate orders per farmer; handle quantity validation	**High**	Implemented

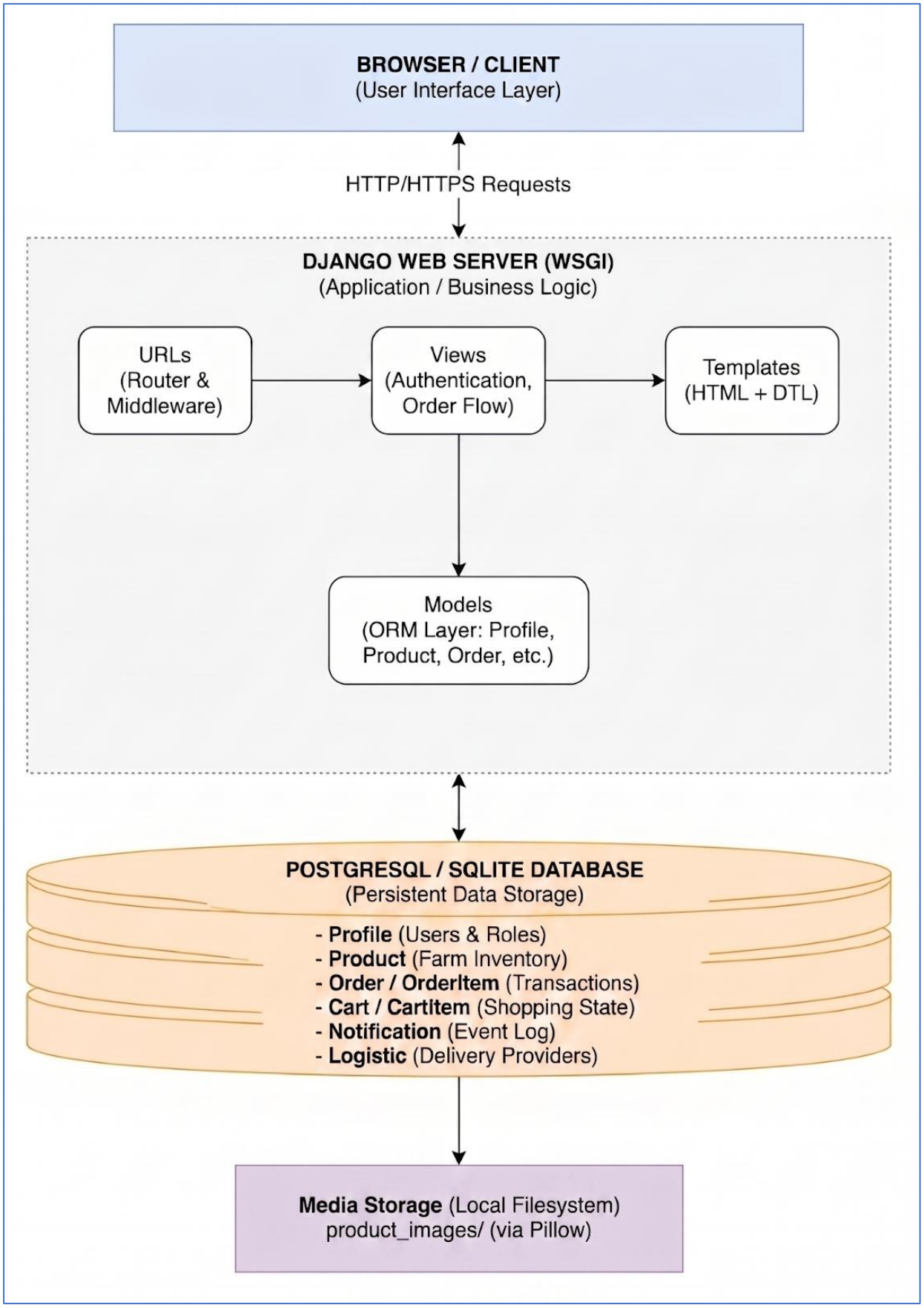
FR-10	Order Status Tracking	Display complete order lifecycle (PENDING → CONFIRMED → DELIVERED → COMPLETED) and enable filtering by status	**High**	Implemented
FR-11	Order Confirmation (Farmer)	Allow farmers to confirm, reject, or cancel pending orders; update order status	**High**	Implemented
FR-12	Logistics Assignment	Enable farmers to assign logistics providers to confirmed orders and add delivery notes	**High**	Implemented
FR-13	Delivery Status Updates	Support status transitions (ASSIGNED → OUT_FOR_DELIVERY → DELIVERED) for order tracking	**High**	Implemented
FR-14	Order Completion (Buyer)	Allow buyers to confirm receipt and mark orders as COMPLETED	**High**	Implemented
FR-15	Notifications System	Generate real-time notifications for order events (new order, confirmation, delivery status)	**High**	Implemented
FR-16	User Profile Management	Allow users to view and update profile details (farm info for farmers, address for buyers)	**Medium**	Implemented
FR-17	Category Management	Provide admin panel for adding/editing/deleting product categories	**Medium**	Implemented
FR-18	Logistics Management	Provide admin panel for managing logistics providers and contact information	**Medium**	Implemented
FR-19	Role-Based Access Control	Restrict farmer views to farmers, buyer views to buyers using decorators and middleware	**High**	Implemented
FR-20	Responsive UI	Ensure platform works on desktop and mobile browsers with consistent styling	**Medium**	Partially Implemented

3.2 Non-Functional Requirements (NFR)

ID	Requirement	Description	Measurable Target
NFR-1	**Performance**	System should handle concurrent users and respond within acceptable time	Page load < 2 seconds
NFR-2	**Security**	Implement user authentication, CSRF protection, SQL injection prevention	All form data validated, HTTPS ready for production
NFR-3	**Scalability**	Database normalized to 3NF; support future feature addition without major refactoring	Supports up to 10K users and 100K products
NFR-4	**Usability**	Intuitive navigation, clear user flows, minimal learning curve	User can complete purchase within 5 clicks
NFR-5	**Maintainability**	Clean, documented code with separation of concerns (Models, Views, Templates)	Code duplication < 10%; cyclomatic complexity < 5 per function
NFR-6	**Data Integrity**	Prevent orphaned records, maintain referential integrity, atomic transactions	All FK relationships with CASCADE/SET_NULL; transaction rollback on checkout failure
NFR-7	**Availability**	System uptime and reliability for continuous operation	Target 99.5% uptime (dev), database backups daily
NFR-8	**Auditability**	Track order status changes, user actions for dispute resolution	All changes timestamped with created_at/updated_at fields

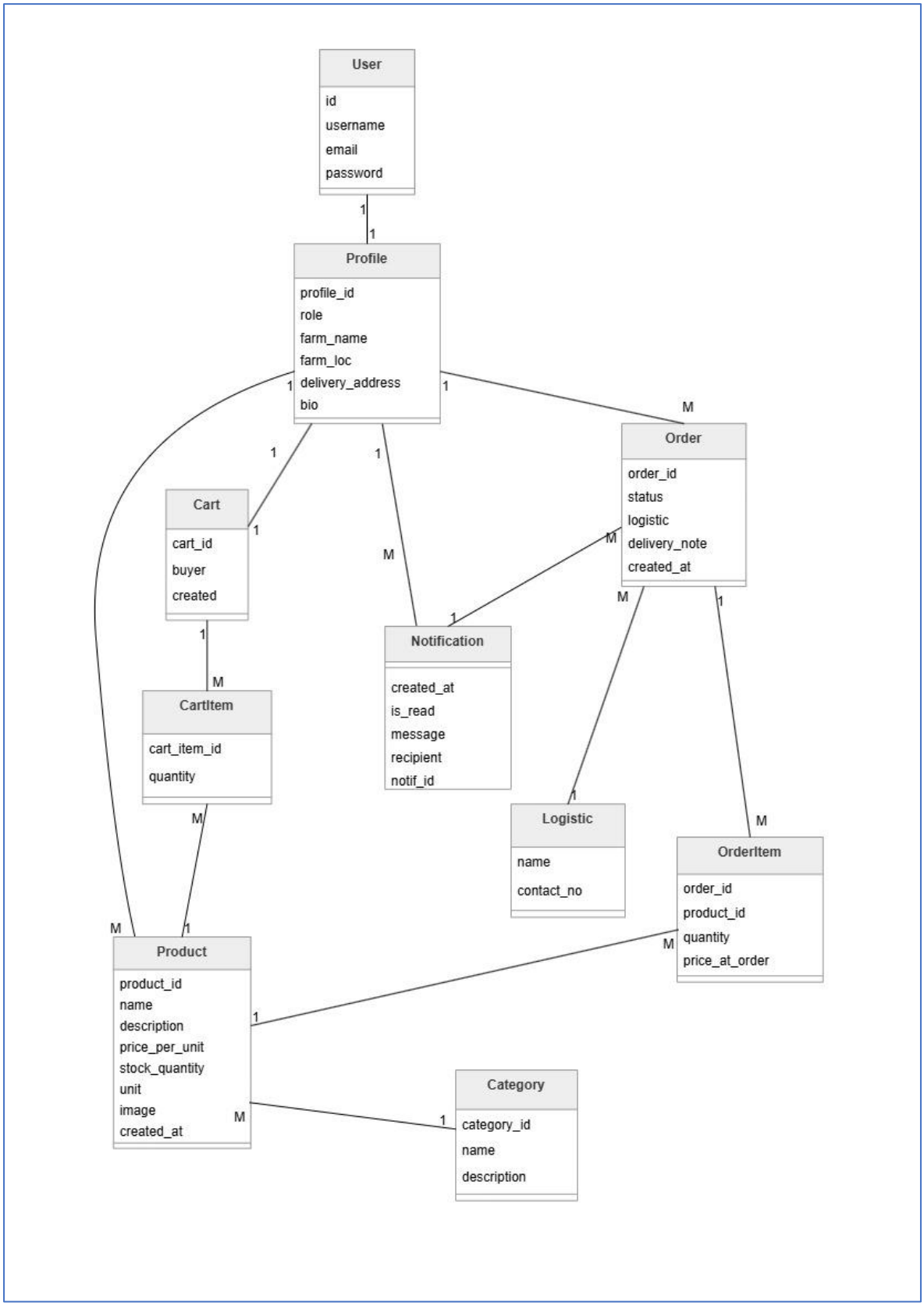
4. System Design & Architecture

4.1 System Architecture Diagram

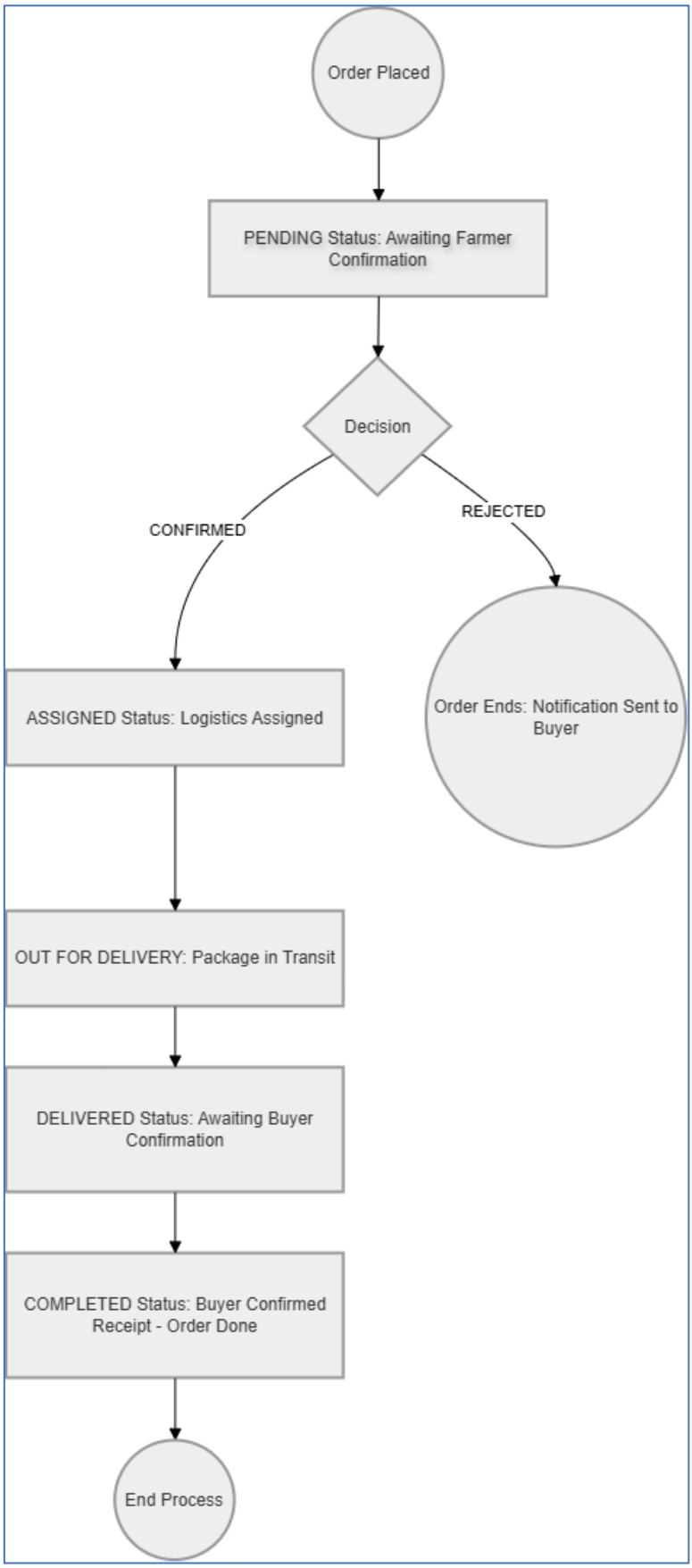


4.2 System Modeling

4.2.1 Use Case Diagram

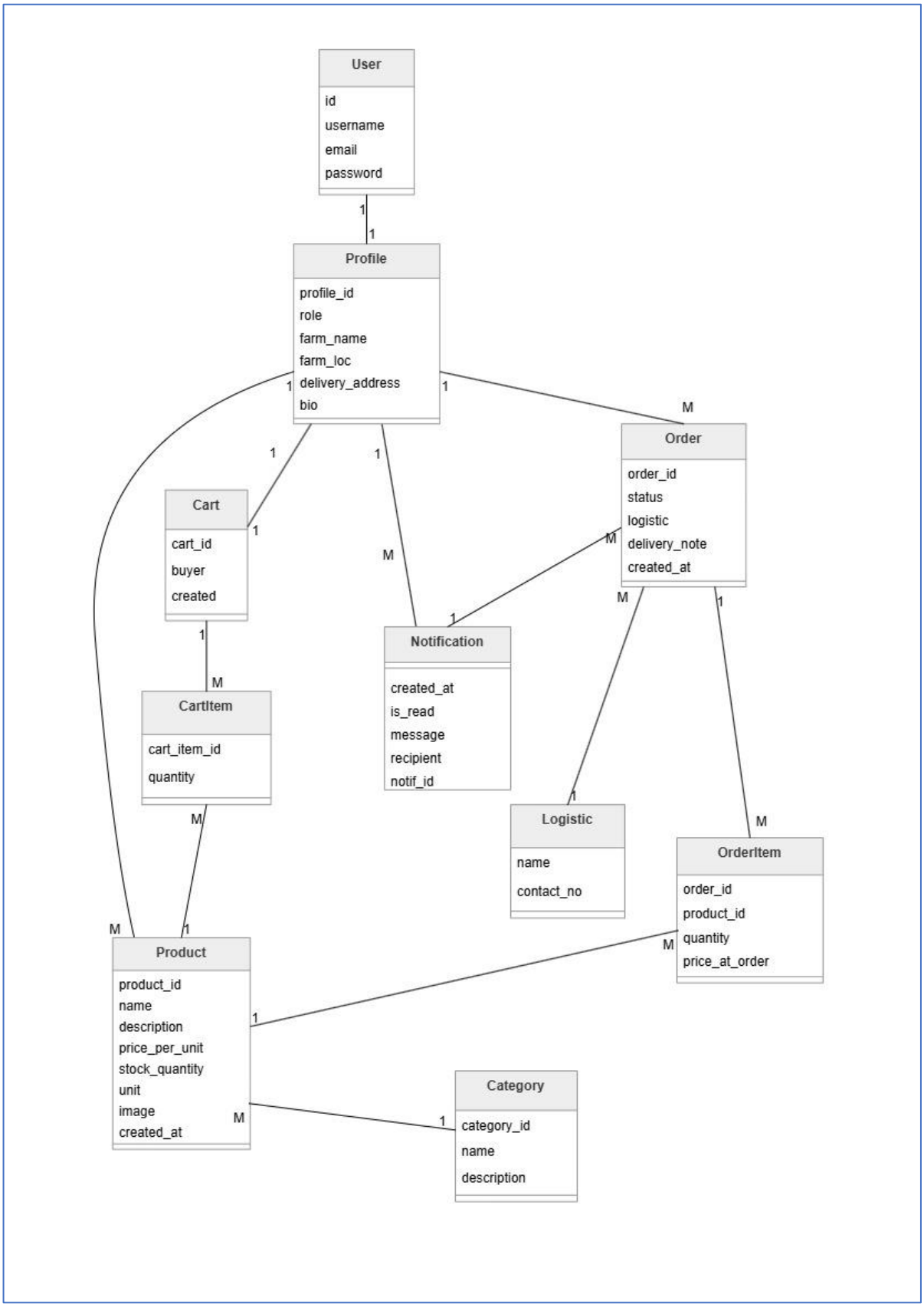


4.2.2 Activity Diagram (Order Lifecycle)



4.3 Database Design:

4.3.1 Entity Relationship Diagram (ERD)



4.3.2 Data Dictionary

Table	Field/Column	Data Type	Constraints	Description
Profile	profile_id	AutoField	PRIMARY KEY	Unique profile identifier
	user_id	OneToOneField	NOT NULL, FK to auth_user	Link to Django User
	role	CharField(20)	CHOICES: farmer/buyer	User role for access control
	farm_name	CharField(255)	NULL, BLANK	Farmer-specific field
	farm_location	CharField(255)	NULL, BLANK	Farmer location for directory
	bio	TextField	NULL, BLANK	Farmer profile description
	delivery_address	TextField	NULL, BLANK	Buyer's delivery location
Category	category_id	AutoField	PRIMARY KEY	Category identifier
	name	CharField(255)	NOT NULL, UNIQUE	Category name (Vegetables)
	description	TextField	NULL, BLANK	Category description
Product	product_id	AutoField	PRIMARY KEY	Product identifier
	farmer_id	ForeignKey	NOT NULL, FK to Profile	Product owner
	category_id	ForeignKey	NOT NULL, FK to Category	Product classification
	name	CharField(255)	NOT NULL	Product name
	description	TextField	NOT NULL	Product details
	price_per_unit	IntegerField	NOT NULL, DEFAULT 0	Unit price in local currency
	stock_quantity	IntegerField	NOT NULL, DEFAULT 0	Available quantity
	unit	CharField(50)	NOT NULL	Measurement unit (kg, piece, l)
	image	ImageField	NULL, BLANK	Product photo
	created_at	DateTimeField	AUTO_NOW_ADD	Timestamp of creation
Cart	cart_id	AutoField	PRIMARY KEY	Cart identifier
	buyer_id	OneToOneField	NOT NULL, FK to Profile	Buyer owning cart
	created_at	DateTimeField	AUTO_NOW_ADD	When cart was created
	updated_at	DateTimeField	AUTO_NOW	Last modification timestamp
CartItem	cart_item_id	AutoField	PRIMARY KEY	Item identifier
	cart_id	ForeignKey	NOT NULL, FK to Cart, CASCADE	Cart reference
	product_id	ForeignKey	NOT NULL, FK to Product	Product being purchased
	quantity	PositiveIntegerField	NOT NULL, DEFAULT 1	Quantity ordered

Order	order_id	AutoField	PRIMARY KEY	Order identifier
	buyer_id	ForeignKey	NOT NULL, FK to Profile	Purchaser
	farmer_id	ForeignKey	NOT NULL, FK to Profile	Seller
	status	CharField(20)	CHOICES: See Status Flow	Order status
	logistic_id	ForeignKey	NULL, BLANK, FK to Logistic, SET_NULL	Assigned logistics
	delivery_note	TextField	NULL, BLANK	Farmer's delivery notes
	created_at	DateTimeField	AUTO_NOW_ADD	Order creation time
	updated_at	DateTimeField	AUTO_NOW	Last status change
OrderItem	order_item_id	AutoField	PRIMARY KEY	Item identifier
	order_id	ForeignKey	NOT NULL, FK to Order, CASCADE	Associated order
	product_id	ForeignKey	NOT NULL, FK to Product	Product ordered
	quantity	PositiveIntegerField	NOT NULL, DEFAULT 1	Quantity
	price_at_order	IntegerField	NOT NULL	Price snapshot at checkout
Notification	notification_id	AutoField	PRIMARY KEY	Notification ID
	recipient_id	ForeignKey	NOT NULL, FK to Profile	Recipient user
	order_id	ForeignKey	NULL, BLANK, FK to Order	Related order (optional)
	message	TextField	NOT NULL	Notification content
	is_read	BooleanField	NOT NULL, DEFAULT False	Read status
	created_at	DateTimeField	AUTO_NOW_ADD	Notification timestamp
Logistic	id	AutoField	PRIMARY KEY	Logistic provider ID
	name	CharField(100)	NOT NULL	Provider name (Pathao)
	contact_number	CharField(20)	NULL, BLANK	Service contact

Status Flow:

PENDING → CONFIRMED → ASSIGNED → OUT_FOR_DELIVERY → DELIVERED → COMPLETED

5. Technical Implementation

5.1 Backend / Data Interaction Documentation

This system is implemented with Django server-side rendering, not REST APIs.

Frontend-backend communication is handled through HTML forms (GET/POST), Django views, ORM operations, and template rendering/redirects.

Method	URL	Purpose	Data Processing / Management
GET, POST	/register/	User registration	Validates input, creates User, then creates role-specific Profile (farmer or buyer).
GET, POST	/login/	User login	Authenticates user, logs session in, merges anonymous session cart into DB cart for buyers.
POST	/logout/	User logout	Clears authenticated session and redirects to home.
GET	/	Home page	Loads categories and renders home template.
GET	/products/	Product browsing	Reads Product and Category; applies search and category filtering using query parameters.
GET, POST	/farmer/dashboard/	Farmer dashboard + add product	Loads farmer products/orders; on POST creates new Product.
GET, POST	/farmer/product/edit/<product_id>/	Edit farmer product	Restricts to product owner; updates product fields and image.
GET, POST	/profile/	Profile management	Updates User fields and role-dependent profile fields.
GET	/buyer/dashboard/	Buyer dashboard	Loads buyer orders, order statistics, and spending summary.
GET	/cart/	View cart	Auth users: DB cart (Cart, CartItem). Guests: session-based cart dictionary.

POST	/cart/add/<product_id>/	Add to cart	Auth users: create/update CartItem; guests: update session cart; enforces stock limit.
POST	/cart/update/<item_id>/	Update cart item	Increase/decrease/remove cart item in DB or session cart.
GET (currently used), state-changing	/checkout/	Place orders	Groups cart items by farmer, creates Order + OrderItem, decrements stock, creates notifications, clears cart.
POST	/order/action/farmer/<order_id>/	Farmer order actions	Handles confirm/reject/assign logistic/dispatch/delivered transitions; creates notifications; restores stock on reject.
POST	/order/action/buyer/<order_id>/	Buyer order actions	Handles cancel/confirm receipt; creates notifications; restores stock on cancel.
Django admin auth flow	/admin/	Admin data management	Uses Django admin for managing registered models.

5.2 Role-Based Access Control (RBAC)

- The system uses role-based authorization via Profile.role plus Django authentication decorators.
- Roles are defined as farmer and buyer in the Profile model.
- Registration assigns role-specific profile data:
  - Farmer: farm_name, farm_location, bio
  - Buyer: delivery_address
- Protected routes use login_required.
- Each dashboard/action view check's role explicitly before allowing access:
  - Farmer-only: farmer dashboard, product edit, farmer order actions
  - Buyer-only: buyer dashboard, cart/checkout, buyer order actions
- Post-login routing is role-aware:
  - Farmer - farmer dashboard
  - Buyer - home page (with buyer features enabled in UI)
- UI visibility is role-driven (navbar links, cart badge, dashboard options).
- "Admin" is not a custom app role; it is Django's built-in admin system (/admin/) with staff/superuser permissions.

5.3 Frontend Organization:

Frontend is organized with Django templates and reusable layout blocks.

- **templates/base.html** is the shared master layout (navbar, messages, scripts, content blocks).
- **templates/F2M/** contains page templates (home, products, dashboards, profiles, auth, cart).
- **templates/F2M/footer.html** is a reusable shared partial.

Reusability approach:

- Template inheritance: pages extend base.html and fill title, content, footer blocks.
- Partial reuse: footer is included where needed.
- Shared context processors provide global reusable data (cart_count, notification flags) to all templates.
- A consistent design system is maintained with shared Tailwind utility conventions.

Atomic Design note:

- A formal Atomic Design structure (atoms/molecules/organisms folders) is not explicitly implemented.
- Current reuse is practical and template-based, with good reuse of shared layouts but some repeated dashboard/profile markup.

6. User Manual

6.1 Getting Started

Step 1: Access the Platform

1. Open your web browser and navigate to http://127.0.0.1:8000/
2. You will see the Farm2Market Homepage with a navigation bar and featured products

Step 2: Register Your Account

For Farmers:

1. Click "Register" in the navigation bar
2. Select "Farmer" from the role dropdown
3. Fill in the required fields:
    a. Username: Your unique identifier
    b. Email: Valid email address
    c. Password: Strong password (min 8 characters)
    d. Confirm Password: Re-enter your password
    e. Farm Name: Name of your farm
    f. Farm Location: Physical location/address of your farm
    g. Bio: Brief description of your farm (optional)
4. Click "Register" button
5. You will be redirected to the login page with a success message

For Buyers:

1. Click "Register" in the navigation bar
2. Select "Buyer" from the role dropdown
3. Fill in the required fields:
    a. Username: Your unique identifier
    b. Email: Valid email address
    c. Password: Strong password (min 8 characters)
    d. Confirm Password: Re-enter your password
    e. Delivery Address: Complete residential address for order delivery
4. Click "Register" button
5. You will be redirected to the login page with a success message

Step 3: Log in to Your Account

1. Click "Login" in the navigation bar
2. Enter your Username and Password
3. Click "Sign In" button
4. You will be redirected to your role-specific dashboard

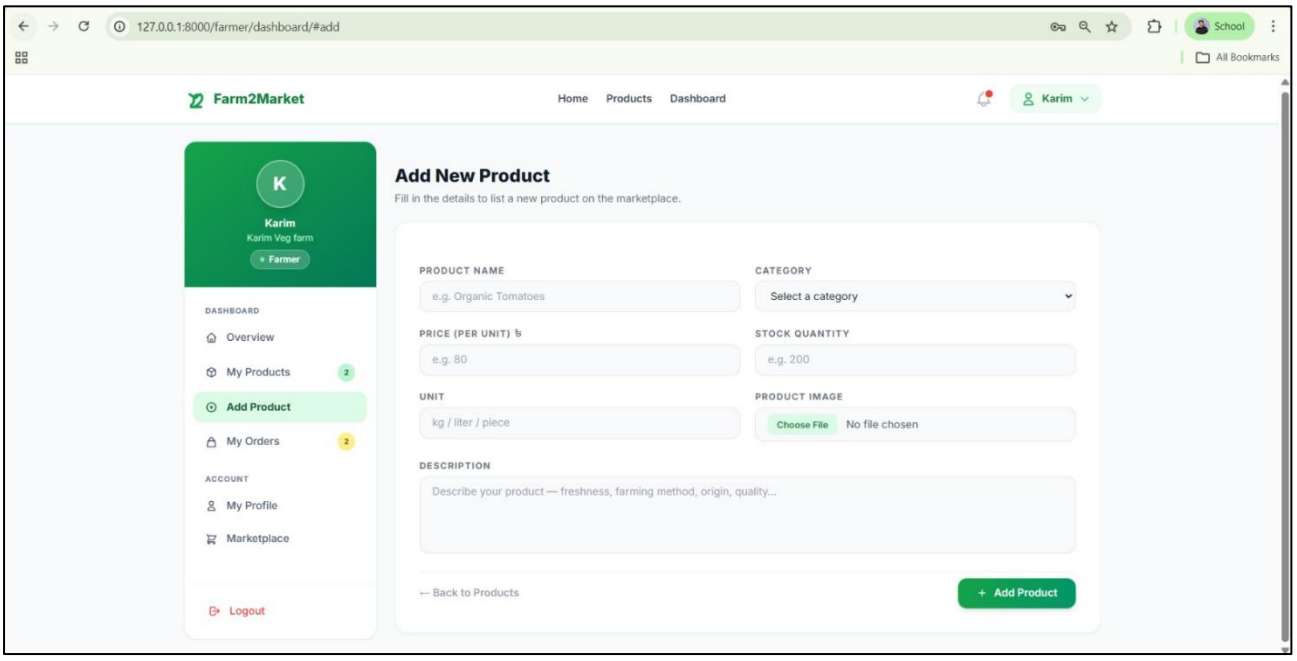
Step 4: Navigate the Dashboard

1. Farmer Dashboard:
    a. View your product listings and inventory
    b. See all orders from buyers
    c. Manage profiles and farm information
2. Buyer Dashboard:
    a. View your order history
    b. Track order statuses
    c. Browse available products

6.2 Feature Walkthroughs

FEATURE 1: Add Product (Farmer)

Screenshot:



Steps:

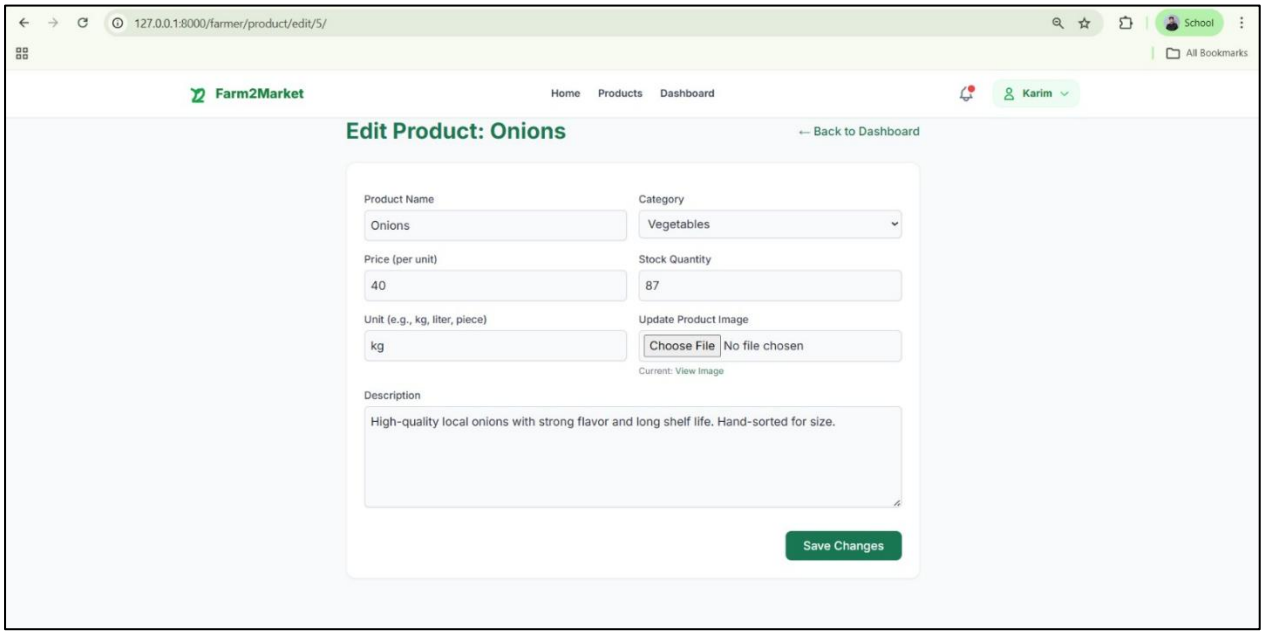
1. Log in as a Farmer
2. Click on the Farmer Dashboard link in the navigation bar
3. Scroll down to the "Add New Product" section
4. Fill in the following fields:
    a. Product Name: ("Organic Tomatoes")
    b. Category: Select from dropdown (Vegetables, Fruits)
    c. Description: Detailed description of the product
    d. Price per Unit: Enter price in local currency (50)

    e. Stock Quantity: Number of units available (100)
    f. Unit: Select measurement (kg, piece, liter)
    g. Product Image: Upload a product photo (JPG/PNG)
5. Click "Add Product" button
6. You will see a success message, and the product will appear in your product list

Expected Result: Product appears in the product list and is visible to all buyers

FEATURE 2: Edit Product (Farmer)

Screenshot:



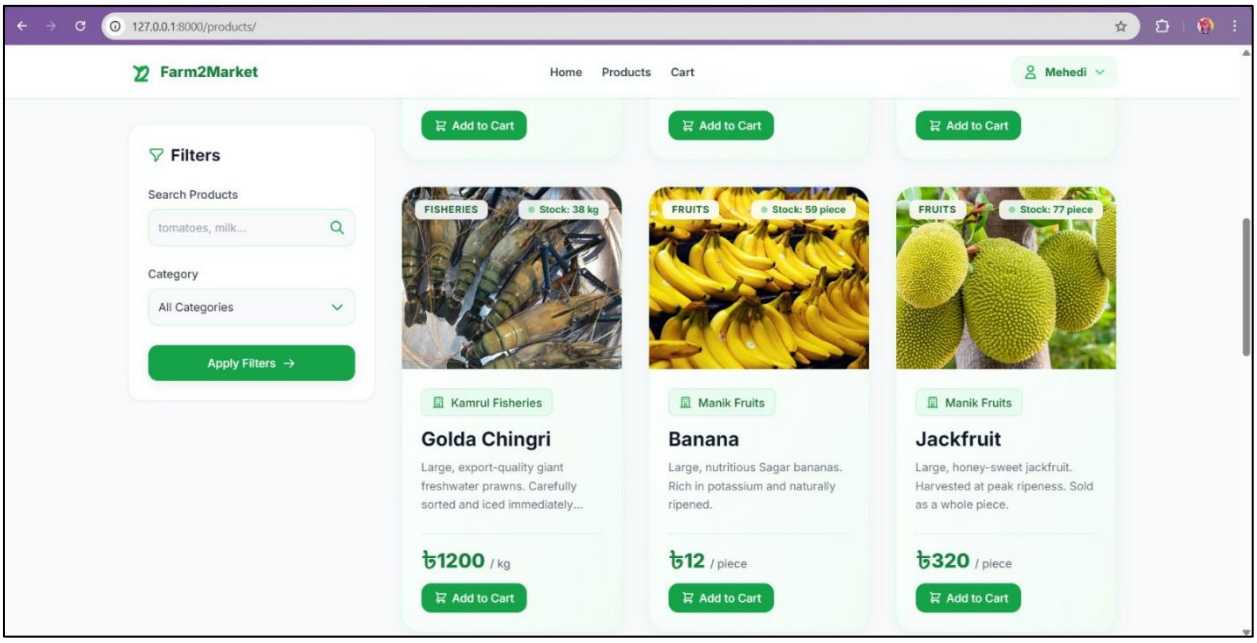
Steps:

1. Log in as a Farmer
2. Go to Farmer Dashboard
3. Find the product you want to edit in your Product List
4. Click the "Edit" button next to the product
5. Modify any of the following fields:
    a. Product name, category, description, price, stock, unit, or image
6. Click "Save Changes" button
7. Success message appears; updates are reflected immediately

Expected Result: Product details are updated; changes visible to buyers immediately

FEATURE 3: Browse Products (Buyer)

Screenshot:



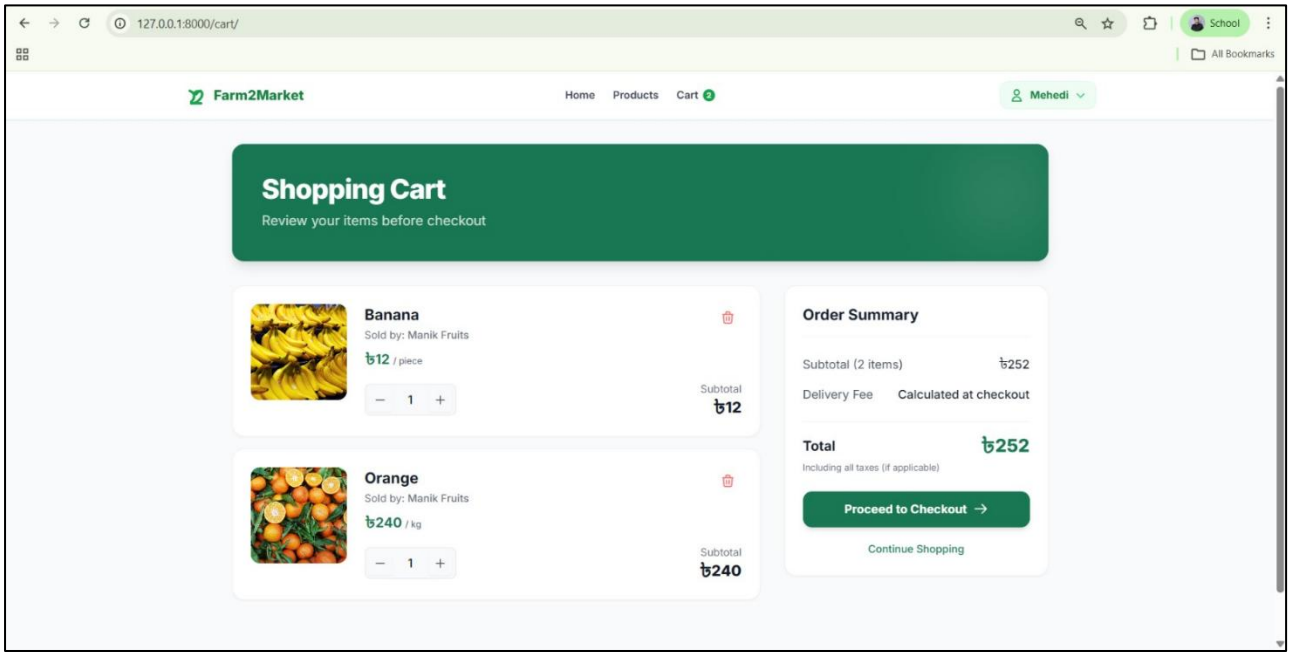
Steps:

1. (Optional) Log in as a Buyer (browsing works for anonymous users too)
2. Click on "Products" in the navigation bar
3. You will see a grid of products from all farmers
4. Optional filters:
    a. Search by Name: Type product name in search box, click "Search"
    b. Filter by Category: Select category from dropdown
    c. View Product Details: Click on any product card to see:
        i. Product image, full description
        ii. Price per unit, available quantity
        iii. Farmer name and farm location
5. Click "Add to Cart" on any product

Expected Result: Products are displayed; you can search, filter, and add to cart

FEATURE 4: Manage Shopping Cart (Buyer)

Screenshot:



Steps:

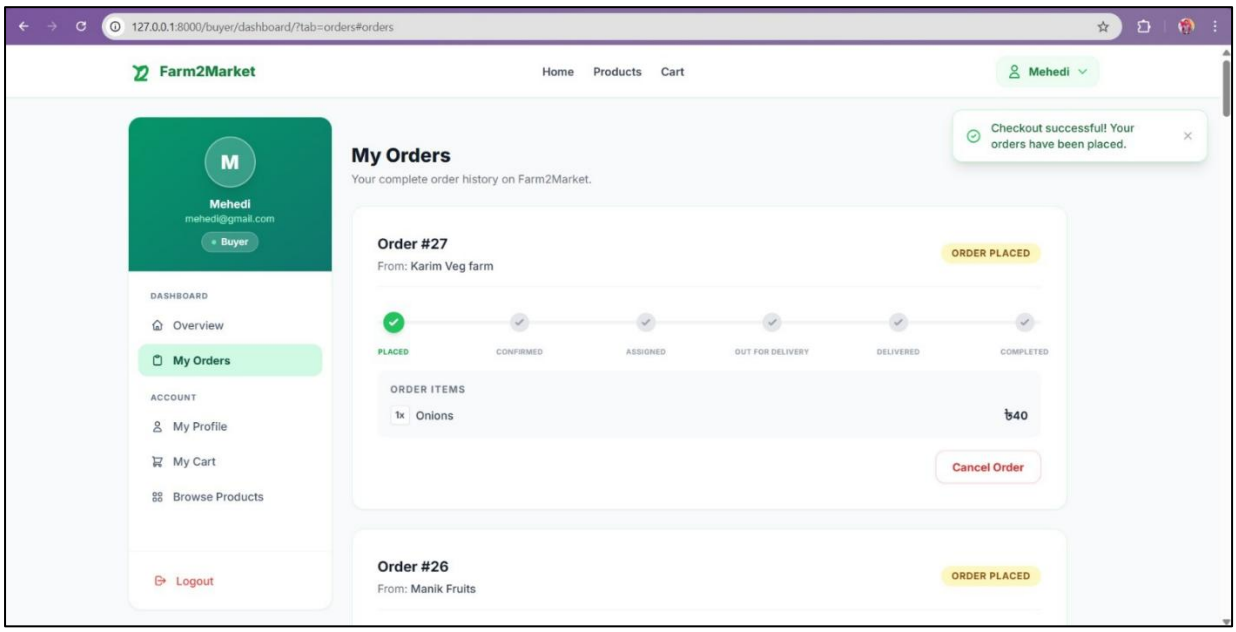
1. Click "View Cart" in the navigation bar (or cart icon)
2. Your shopping cart appears showing:
    a. Product name, image, unit price
    b. Current quantity (can be modified)
    c. Subtotal per item
    d. Total cart value
3. To modify quantities:
    a. Change the number in the Quantity field
    b. Click "Update" button
4. To remove an item:
    a. Set quantity to 0 and click "Update"
5. Click "Proceed to Checkout" button

Note: If you're not logged in, your cart is saved in your browser session and will merge into your account when you log in.

Expected Result: Cart contents displayed; quantities can be modified; checkout proceeds

FEATURE 5: Checkout & Place Order (Buyer)

Screenshot:



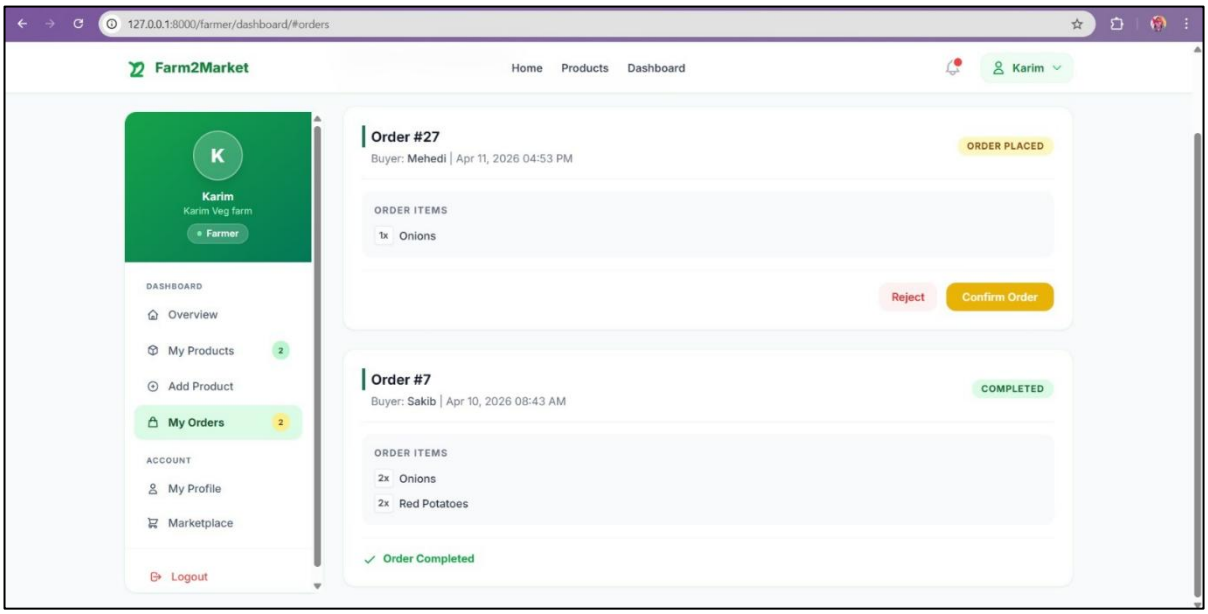
Steps:

1. From your cart, click "Proceed to Checkout"
2. Review your order details:
    a. Items are automatically grouped by farmer
    b. Each farmer's products form a separate order
    c. Shows item count, unit prices, and subtotal per farmer
3. Verify your Delivery Address (shown at top)
4. Click "Confirm Purchase" button
5. Orders are created instantly; you'll see a confirmation message
6. You'll be redirected to your Buyer Dashboard

Expected Result: Orders created; cart cleared; buyer notified of confirmation; farmer receives notifications

FEATURE 6: Manage Orders (Farmer)

Screenshot:



Steps:

1. Log in as a Farmer
2. Go to Farmer Dashboard
3. Scroll to "Recent Orders" section
4. You'll see all incoming orders with status badges (PENDING, CONFIRMED, etc.)

To Confirm an Order:

1. Click on a PENDING order
2. Review buyer details, items, quantities, and delivery address
3. Click "Confirm Order" button
4. Status changes to CONFIRMED
5. Buyer receives a notification

To Assign Logistics:

1. On a CONFIRMED order, click "Assign Logistics"
2. Select a logistics provider from the dropdown (Pathao, Steadfast)
3. (Optional) Add delivery notes/instructions
4. Click "Assign & Dispatch" button
5. Status changes to ASSIGNED, then OUT_FOR_DELIVERY
6. Buyer is notified with logistics provider info

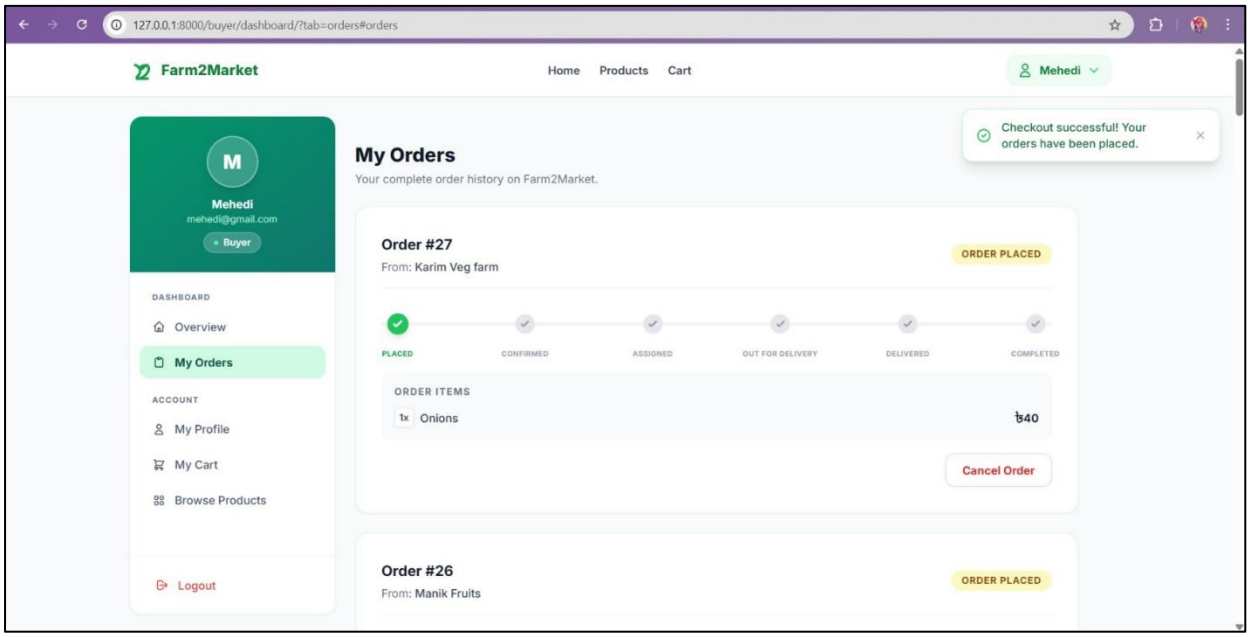
To Reject an Order:

1. Click on a PENDING order
2. Click "Reject Order" button (if you can't fulfill it)
3. Status changes to REJECTED
4. Buyer receives notification to contact you

Expected Result: Order status updates; buyer and farmer notifications sent; logistics information tracked

FEATURE 7: Track Orders (Buyer)

Screenshot:



Steps:

1. Log in as a Buyer
2. Click "My Orders" or "Buyer Dashboard"
3. All your orders appear, grouped by status:
    a. PENDING: Awaiting farmer confirmation
    b. CONFIRMED: Order confirmed, logistics being assigned
    c. OUT_FOR_DELIVERY: Package in transit
    d. DELIVERED: Package arrived, waiting for your confirmation
    e. COMPLETED: Order finished
    f. REJECTED/CANCELLED: Order not fulfilled
4. Click on any order to see:
    a. Order ID, creation date, last update
    b. Items ordered with quantities and prices
    c. Current status with timeline
    d. Assigned logistics provider (if applicable)
    e. Farmer contact info and farm location

To Confirm Receipt (Mark as Completed):

1. Find an order with status "DELIVERED"
2. Click "Confirm Receipt" button
3. Status changes to "COMPLETED"
4. Farmer is notified of successful delivery
5. Order moves to completed history

Expected Result: Order status tracked in real-time; buyer can confirm receipt to complete transaction

FEATURE 8: Notifications

Screenshot:

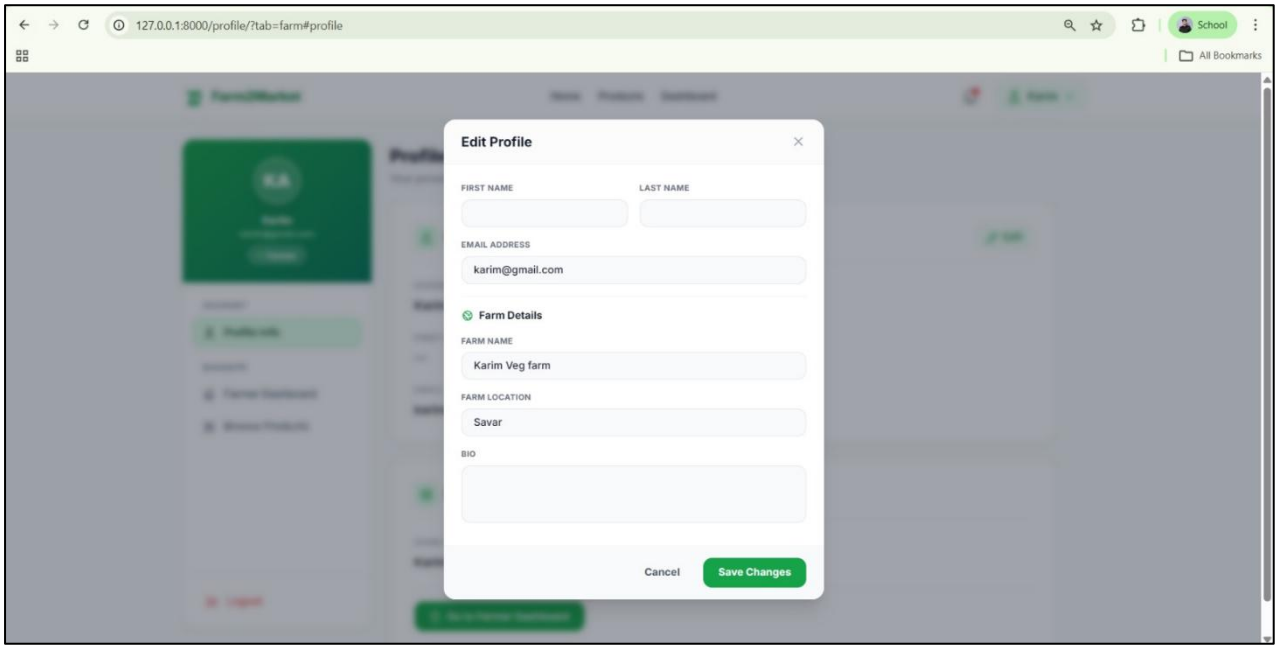
Where Notifications Appear:

1. Navbar Badge: Small red badge shows unread notification count
    a. For farmers: "2 New Orders!" badge
    b. For buyers: "1 Delivered Order" badge
2. Click Notification Badge → View All Notifications:
    a. List of recent notifications with timestamps
    b. Messages sorted by newest first
    c. Mark as read/unread
3. Notification Types:
    a. For Farmer: "New order from [Buyer Name]", "Order confirmed", etc.
    b. For Buyer: "Your order confirmed by [Farmer Name]", "Package out for delivery", etc.

Expected Result: Real-time alerts keep users informed of order status changes

FEATURE 9: Edit Profile

Screenshot:



Steps:

1. Log in to your account
2. Click "Profile" in the navigation bar
3. You'll see your profile information:
4. For Farmers: Farm name, farm location, bio
5. For Buyers: Delivery address
6. Click "Edit" button next to any field
7. Modify the information
8. Click "Save" button
9. Success message appears; changes are saved

Expected Result: Profile information updated; reflected immediately in system

7. Quality Assurance & Testing

7.1 Test Case Scenarios:

Test ID	Test Description	Input Data	Expected Result	Actual Result	Status
T-1	Register as Farmer with valid data	username: john_farmer email: john@farm.com, password: SecurePass123, farm_name: Green Farm, farm_location: Village XYZ, bio: Organic farming	User created, Profile created with role='farmer', Redirected to login	As expected	PASS
T-2	Register Farmer with missing farm_name	username: john2, email: john2@farm.com, password: SecurePass123, farm_name: (empty), farm_location: Village XYZ	Error message: "Farm name and location are required"	Error shown; user not created	PASS
T-3	Register with duplicate username	username: john_farmer (already exists), email: new@test.com, password: SecurePass123	Error: "Username already exists"	Correct error; registration blocked	PASS
T-4	Register with password mismatch	username: tom, email: tom@test.com, password: Pass123, confirm_password: Pass124	Error: "Passwords are required and must match"	Error shown; registration failed	PASS
T-5	Register as Buyer with valid data	username: alice_buyer, email: alice@test.com, password: SecurePass123, delivery_address: 123 Main St, City	User and Profile('buyer') created; Redirected to login	As expected	PASS
T-6	Login with correct credentials	username: john_farmer, password: SecurePass123	User authenticated; session created; Redirected to dashboard	Correct; farmer dashboard loaded	PASS
T-7	Login with incorrect password	username: john_farmer, password: WrongPassword	Error: "Invalid username or password"	Error shown; login failed	PASS
T-8	Farmer adds product	farmer_id: john_farmer, name: "Ripe Tomatoes", price: 50, stock: 100, category: Vegetables, unit: kg, image: tomato.jpg	Product created, visible in farmer dashboard, queryable by buyers	Product appears in product list	PASS
T-9	Farmer edits product	product_id: 5, new_price: 55, new_stock: 95	Product fields updated, changes visible to buyers immediately	Product updated; prices reflect change	PASS
T-10	Buyer browses products	No input (GET request)	All products from all farmers displayed in grid	Products displayed with farmer info	PASS

T-11	Buyer searches products by name	search_query: "tomato"	Only products with "tomato" in name returned	Filtered results shown correctly	PASS
T-12	Buyer filters by category	category_id: 1 (Vegetables)	Only products in Vegetable's category shown	Category filter works; correct products	PASS
T-13	Add product to anonymous cart	product_id: 5, quantity: 2	Item stored in session['cart']; cart count badge updated	Session cart created; badge shows 1 item	PASS
T-14	Login buyer merges session cart	session_cart: {5: 2}, user_id: alice_buyer	CartItem(product_5, qty=2) created in buyer's Cart object; session cart cleared	Cart merged correctly; items in database	PASS
T-15	Update cart item quantity	cart_item_id: 3, new_quantity: 5	CartItem quantity updated; subtotal recalculated	Quantity changed; total updated	PASS
T-16	Remove item from cart	cart_item_id: 3, quantity: 0	CartItem deleted; cart count updated	Item removed from cart	PASS
T-17	Checkout with items from multiple farmers	cart: {product_1 (farmer_1): qty 2, product_3 (farmer_2): qty 1}	Two Order objects created (1 per farmer); OrderItem records created with price snapshot; cart cleared	Multiple orders created; grouped correctly	PASS
T-18	Checkout empty cart	cart: {}	Error message: "Your cart is empty"	Error shown; checkout prevented	PASS
T-19	Farmer views pending orders	farmer_id: john_farmer	Only orders where farmer_id=john_farmer and status=PENDING shown	Correct orders filtered	PASS
T-20	Farmer confirms order	order_id: 10, action: confirm	Order.status → CONFIRMED; Notification created for buyer	Status changed; notification sent	PASS
T-21	Farmer rejects order	order_id: 10, action: reject	Order.status → REJECTED; Notification: "Order rejected" sent to buyer	Status changed to REJECTED	PASS
T-22	Farmer assigns logistics	order_id: 10, logistic_id: 2 (Pathao), delivery_note: "Leave at door"	Order.logistic_id updated; status → ASSIGNED → OUT_FOR_DELIVERY; Buyer notified	Logistics assigned; notifications sent	PASS
T-23	Buyer views delivered order	order_id: 10, status: DELIVERED	"Confirm Receipt" button **visible**	Button appears; clickable	PASS
T-24	Buyer confirms receipt	order_id: 10, action: confirm_receipt	Order.status → COMPLETED; Notification sent to farmer: "Order completed"	Status → COMPLETED; notification sent	PASS

T-25	Unauthorized farmer access to buyer dashboard	farmer_id: john_farmer, endpoint: /buyer/dashboard/	Access denied; error message: "You must be a buyer to access this page."	Access blocked; redirected to home	PASS
T-26	Unauthorized buyer access to farmer dashboard	buyer_id: alice_buyer, endpoint: /farmer/dashboard/	Access denied; error message: "You must be a farmer to access this page."	Access blocked; redirected to home	PASS
T-27	RBAC: Buyer cannot edit other farmer's product	buyer_id: alice, endpoint: /farmer/product/edit/5/	Access denied (not farmer)	Blocked; error shown	PASS
T-28	RBAC: Farmer cannot edit other farmer's product	farmer_id: john, product_id: 15 (owned by bob), action: edit	Edit prevented; error: "You don't have permission to edit this product"	Product protection enforced	PASS
T-29	View notification badges	user_id: john_farmer, unread_notifications: 3	Badge shows "3" next to notification icon in navbar	Badge displays correctly	PASS
T-30	Notification triggers on new order	order created, order.farmer_id: john	Notification (recipient=john_profile, message="New order from Alice", is_read=False) created	Notification appears immediately	PASS
T-31	Database integrity: Delete farmer cascades products	farmer_id: bob (with 5 products), action: delete	All Product records with farmer_id=bob deleted; OrderItems referencing those products handled	Cascade delete works; data consistency	PASS
T-32	Database integrity: Delete logistic sets order.logistic=NULL	logistic_id: 1 (Pathao), action: delete	All Order.logistic_id=1 → SET_NULL (now NULL); Orders not deleted	Logistic deleted; orders preserved with NULL logistic	PASS
T-33	Stock validation: Cannot add more to cart than in stock	product_stock: 10, cart_quantity_requested: 15	Warning/Error: "Only 10 units available"	Quantity capped or error shown	PASS
T-34	Admin: Add product category	category_name: "Dairy Products", description: "Milk, Cheese, etc."	Category created, visible in dropdown on product add/edit forms	Category appears in dropdowns	PASS
T-35	Admin: Add logistics provider	logistic_name: "Steadfast", contact: "01234567890"	Logistic created; visible in farmer's logistics dropdown	Logistic available for assignment	PASS
T-36	Session timeout: Anon cart persists across page reloads	cart: {5: 2}, action: reload page	Session cart [product_5: qty 2] still present	Cart persists in session	PASS
T-37	XSS Protection: HTML injection in product description	description: ""	Script tags escaped; displayed as text, not executed	HTML entity encoded; safe display	PASS

T-38	CSRF Protection: Form submission without CSRF token	POST /checkout/ without csrf token	Request rejected; error: "CSRF token missing or incorrect"	CSRF blocked unauthorized request	PASS
T-39	SQL Injection Prevention: Malicious search query	search: "tomato' OR '1'='1"	Query parameterized; treated as literal string; no injection	Safe; ORM prevents injection	PASS
T-40	Mobile Responsiveness: Browse on mobile viewport (375px)	action: load /products/ on mobile browser	Layout responsive; navigation collapses to hamburger; products in single column	Responsive design works	PASS

7.2 Requirements Traceability Matrix (RTM):

Requirement ID	Feature	Test ID	Test Description	Coverage
FR-1	User Registration	T-1, T-2, T-3, T-4, T-5	Farmer registration, validation checks, duplicate prevention, buyer registration	100%
FR-2	User Authentication	T-6, T-7	Login with correct credentials, login with incorrect password	100%
FR-3	Product Listing (Farmer)	T-8	Farmer adds new product with all fields	100%
FR-4	Product Management	T-9	Farmer edits product details	100%
FR-5	Product Browsing (Buyer)	T-10, T-11, T-12	Browse all products, search by name, filter by category	100%
FR-6	Shopping Cart	T-13, T-15, T-16, T-33	Add to cart, update quantity, remove items, stock validation	100%
FR-7	Cart Management	T-13, T-15, T-16	Add items, modify quantities, remove items	100%
FR-8	Cart Merge on Login	T-14	Session cart merges into database cart upon login	100%
FR-9	Checkout Process	T-17, T-18	Multiple farmer grouping, empty cart prevention	100%
FR-10	Order Status Tracking	T-19, T-23, T-24	View orders, track delivery status, view completed orders	100%
FR-11	Order Confirmation (Farmer)	T-20, T-21	Confirm order, reject order	100%
FR-12	Logistics Assignment	T-22	Farmer assigns logistics provider and notes	100%
FR-13	Delivery Status Updates	T-22, T-24	Order status transitions (ASSIGNED → OUT_FOR_DELIVERY → DELIVERED → COMPLETED)	100%
FR-14	Order Completion (Buyer)	T-24	Buyer confirms receipt to mark order COMPLETED	100%
FR-15	Notifications System	T-29, T-30	Notification badges, triggers on order events	100%
FR-16	User Profile Management	T-12	Profile viewing and editing	100%
FR-17	Category Management	T-34	Admin adds product categories	100%
FR-18	Logistics Management	T-35	Admin adds logistics providers	100%
FR-19	Role-Based Access Control	T-25, T-26, T-27, T-28	Farmer/buyer dash separation, edit permission checks	100%
FR-20	Responsive UI	T-40	Mobile responsiveness testing	Partial
NFR-1	Security	T-37, T-38, T-39	XSS protection, CSRF protection, SQL injection prevention	100%
NFR-2	Data Integrity	T-31, T-32	Cascade deletes, referential integrity	100%

8. Individual Contribution

Md. Shardul Rahman Turjo - Backend Logic

Technical Contribution

- Implemented authentication system: registration, login, logout, and role-based redirect (farmer/buyer)
- Developed farmer dashboard for product management, stock tracking, and order handling
- Built product edit feature with ownership validation
- Created product listing with search and category filtering
- Designed cart system (DB cart for users, session cart for guests) with stock validation and subtotal calculation
- Implemented checkout and order system: grouped items by farmer, created orders, deducted stock, and sent notifications
- Developed profile view with role detection, updates, and farmer statistics
- Configured 15 URL routes for core application features
- Worked mainly in: views.py, urls.py, admin.py

Technical Challenges

- Implementing secure role-based access using login_required and profile checks
- Handling incomplete or invalid registrations and cleaning up data
- Managing stock validation across cart and order processes

Learning Outcomes

- Learned to use related_name for better model relationships
- Gained experience in role-based authorization without external libraries
- Improved use of Django messages for user feedback

Chowdhury Nabil Ahmed - Frontend (UI/UX)

Technical Contribution

- Designed base layout with navbar, role-based menus, and notification messages
- Built home page with hero section and call-to-action
- Created dynamic registration form based on user role
- Developed product listing UI with search and filtering
- Designed farmer dashboard UI for product and order management
- Implemented cart page with quantity control and summary
- Built separate profile pages for farmers and buyers
- Developed additional pages: login, edit product, buyer dashboard
- Worked mainly in multiple HTML templates (base, home, register, products, dashboard, cart, profile pages)

Technical Challenges

- Maintaining consistent UI across multiple templates
- Passing and displaying Django context data properly
- Making dashboards and pages responsive
- Correct use of template tags and conditional rendering

Learning Outcomes

- Learned template inheritance (extends, block)
- Improved role-based UI rendering using Django context
- Built reusable components and shared layouts
- Maintained consistent UI design patterns

Md. Sajib Ahammad - Database

Technical Contribution

- Converted ERD into Django models
- Created 9 models: Profile, Category, Product, Cart, CartItem, Logistic, Order, OrderItem, Notification
- Defined relationships using OneToOneField and ForeignKey with related_name
- Set up database schema using migrations
- Worked mainly in: models.py, migrations/0001_initial.py

Technical Challenges

- Designing correct relationships between users, carts, orders, and products
- Ensuring referential integrity with proper on_delete rules
- Maintaining data consistency for stock updates and order flow

Learning Outcomes

- Learned how to convert ER diagrams into relational models
- Understood when to use OneToOne vs ForeignKey
- Improved use of related_name for reverse queries
- Gained experience with Django migrations for schema management

9. Conclusion & Future Work

9.1 Conclusion:

Project Success Summary

Farm2Market solves the problem of middlemen by directly connecting farmers and buyers through a digital platform.

- **Strong System Design:** Built with Django, supports multiple users (farmer & buyer) with secure login and role-based access.
- **Complete Features:** Includes product management, cart, order tracking, delivery process, and notifications
- **Data Security:** Protected against common risks (CSRF, SQL injection, XSS) with proper access control
- **Easy to Use:** Simple dashboards, smooth shopping experience, and real-time order updates
- **Scalable System:** Clean and modular code, easy to expand in the future with good test coverage

Impact & Value

- Farmers earn more by selling directly
- Buyers get fair prices and clear product info
- Overall system makes the supply chain faster and more efficient

9.2 Future Enhancements (Version 2.0)

1. Payment & Transaction System

- Integrate Stripe, bKash, and Nagad for secure payments
- Support card, mobile wallet, and cash-on-delivery
- Track transaction history and payment status

2. Smart Search & Product Discovery

- Filter by price, farmer rating, and availability
- Sort by popularity, newest, and best deals
- Auto-complete search with suggestions

3. Notifications & User Communication

- SMS alerts for farmers on new orders
- Email confirmations for buyers
- Delivery updates via SMS notifications

4. Analytics & Dashboard Insights

- Farmers: sales, revenue trends, product performance
- Buyers: order history, spending patterns
- Admin: GMV, growth metrics, total orders

5. Mobile App & User Experience

- Native apps using React Native or Flutter
- Push notifications for real-time updates
- Optimized mobile checkout experience

10. Appendices

Appendix A:

GitHub Repository: https://github.com/nabil0203/Farm2Market

Appendix B:

Installation & Setup Guide:

Prerequisites

- Python 3.8+
- pip (Python package manager)
- Git
- PostgreSQL or SQLite (for database)

Step 1: Clone the Repository

```
git clone https://github.com/turjo25/Farm2Market.git
cd Farm2Market
```

Step 2: Create and Activate Virtual Environment

Windows:

```
py -m venv my_env
my_env\Scripts\activate
```

macOS/Linux:

```
python3 -m venv my_env
source my_env/bin/activate
```

Step 3: Navigate to Project Directory

```
cd Farm2Market
```

Step 4: Install Dependencies

```
pip install -r requirements.txt
```

Step 5: Configure Environment Variables

Copy example file

```
copy .env.example .env        # Windows
cp .env.example .env          # macOS/Linux
```

- Edit .env with your configuration
- For local SQLite development, you can leave DATABASE_URL empty

Sample .env:

```
DEBUG=True
SECRET_KEY=your-super-secret-key-here-min-50-chars
DATABASE_URL=
```

Step 6: Download Media Files

Download the product images folder from:
https://drive.google.com/drive/folders/1Ey32ILIr1SpPVf5Stu5ts3m76XvlxsIs

Extract to: Farm2Market/media/

Step 7: Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 8: Create Superuser (Admin Account)

```
python manage.py createsuperuser
```

- Follow prompts to create admin username, email, password

Step 9: Create Categories & Logistics (via Admin)

```
python manage.py runserver
```

- Visit http://127.0.0.1:8000/admin/
- Login with superuser credentials
- Add Categories: Vegetables, Fruits, Grains, etc.
- Add Logistics: Pathao, Steadfast, Uber, etc.

Step 10: Run the Application

```
python manage.py runserver
```

Open browser: http://127.0.0.1:8000/